

Full-Stack Productivity Platform

An Authenticated MERN Productivity Workspace for Tasks, Projects, Goals, Notes, Reminders, and Planning

Author: Keeno Smith

Project Type: Full-Stack Software Engineering Project

Platform: Web-based Single-Page Application (SPA)

Implementation: React / Vite frontend + Node.js / Express REST API + MongoDB / Mongoose persistence

Primary Objective: Engineer a persistent, authenticated multi-domain productivity platform with secure user-scoped resource management, interconnected application data, account lifecycle controls and cloud deployment.

Technical Executive Summary

Full-stack software engineering project involving a React SPA, Node.js/Express REST API, MongoDB/Mongoose persistence, JWT authentication, bcrypt password security, authenticated resource access, file processing, multi-model data operations, interconnected domain entities, account lifecycle management and cloud deployment.

The system was engineered as a multi-domain application rather than a collection of isolated frontend views. Its architecture separates presentation, client-side application state, API communication, backend request handling, authentication middleware and persistent data models, providing a complete software path from browser interaction through authenticated API execution to database persistence.

The implementation demonstrates practical capability across the full application stack:

React SPA → Application State → Service/API Layer → Express REST API → Authentication Middleware → Mongoose Models → MongoDB

The platform further incorporates user-scoped resource access, persistent session restoration, credential verification, account and workspace lifecycle operations, concurrent multi-resource retrieval, multipart file processing, environment-aware configuration and production deployment workflows.

From an engineering perspective, the project demonstrates the ability to design and integrate multiple software layers into a coherent application architecture, with particular emphasis on full-stack integration, authenticated data access, document-oriented data modelling, API-driven application design and separation of concerns.

Sector / Domain

Productivity Software / SaaS-style Web Application

The application sits within the productivity and personal-workspace domain, with functionality spanning task management, project organization, goal tracking, reminders, notes, scheduling and user account management.

Its architecture also provides experience relevant to broader business applications where multiple domain entities must be persisted, related to users, exposed through APIs and coordinated through a single frontend.

Key Role

Full-Stack Software Engineer

Responsible for the architecture and implementation of the application across the frontend, backend, database and development layers.

The project provided practical experience designing a complete application from the user interface through to authenticated API endpoints and persistent database storage.

Responsibilities

- Designed and developed the React frontend application.
- Implemented client-side routing and protected application routes.
- Built reusable components for task management, modals, menus, overlays and account workflows
- Designed and consumed REST APIs using the Fetch API.
- Developed Node.js and Express backend routes
- Designed MongoDB/Mongoose schemas for application entities
- Implemented JWT-based authentication and authenticated API access
- Implemented password hashing and password verification with bcrypt
- Implemented user-scoped data access across application resources
- Built CRUD workflows for core productivity entities
- Implemented account management and workspace lifecycle operations.
- Implemented profile management including avatar upload and removal
- Implemented search, sorting and filtering functionality
- Implemented client-side state synchronization following API mutations
- Implemented data export functionality
- Configured environment-dependent API endpoints
- Worked with Git-based source control and cloud deployment workflows.

Debugged integration issues across the frontend, API and database layers.

Non-Technical Project Overview

A full-stack productivity platform engineered to bring multiple personal-workflow systems into a single authenticated application.

The platform combines tasks, projects, goals, notes, reminders, alarms, calendar functionality, notifications, search and account management within one cohesive workspace. Rather than building isolated CRUD pages, the application was designed around interconnected productivity entities, persistent user data and authenticated access.

The system demonstrates end-to-end software engineering across the frontend, backends, database and deployment layers, including:

- React application architecture
- Client-side routing and protected routes
- REST API development and consumption
- MongoDB data persistence through Mongoose
- JWT-based authentication
- Password hashing and verification
- User-specific data access
- CRUD operations across multiple domain entities
- Complex React state management
- Reusable component architecture
- Search, filtering and sorting
- Data export and destructive account operations
- File uploads for profile avatars
- Cloud deployment and environment-based configuration

The project is representative of a complex independently engineered full-stack web application rather than a frontend-only demonstration

System Architecture

The application follows a layered full-stack architecture in which each major responsibility is separated into an appropriate application layer.

Application

- └─ Presentation Layer
 - └─ React SPA
 - └─ React Router
 - └─ Reusable Application Components

Application State Layer

- └─ React State
- └─ React Context
- └─ Authentication State

Service / Integration Layer

- └─ REST API Services
 - └─ Fetch API
 - └─ Bearer Authentication
 - └─ JSON / FormData Requests

API Layer

- └─ Node.js
- └─ Express
- └─ Route Handlers
- └─ Authentication Middleware

Persistence Layer

- └─ Mongoose Models
- └─ Schema Definitions
- └─ ObjectId Relationships
- └─ User-Scoped Queries

Data Layer

- └─ MongoDB

The resulting execution path can be represented as:

User Interaction

- React Application
- Application State
- Service Layer
- HTTP Request
- Express Route
- Authentication Middleware
- Domain Operation
- Mongoose Model
- MongoDB
- API Response
- Client State Reconciliation
- Rendered Application State

This architecture establishes a clear separation between presentation, application behaviour, transport, server-side processing and persistence. The frontend does not communicate directly with MongoDB; all persistent operations pass through authenticated backend APIs.

Engineering Scope

The implementation spans the complete software application lifecycle, from client-side application architecture through persistent backend storage and deployment.

Engineering Scope

- ├ Client Application
 - ├ SPA architecture
 - ├ Routing
 - ├ Authentication state
 - ├ Application state
 - └ Domain-oriented interfaces
- ├ **API & Application Services**
 - ├ REST endpoint design
 - ├ HTTP operations
 - ├ Authentication middleware
 - ├ Resource operations
 - └ Account lifecycle operations
- ├ **Data Architecture**
 - ├ MongoDB persistence
 - ├ Mongoose schemas
 - ├ User ownership
 - ├ Cross-entity references
 - └ Multi-model operations
- ├ **Security**
 - ├ JWT authentication
 - ├ Bearer-token authorization
 - ├ bcrypt credential protection
 - └ Sensitive-operation verification
- └ **Delivery**
 - ├ Environment configuration
 - ├ Production builds
 - ├ Git / GitHub
 - └ Cloud deployment

The breadth of implementation demonstrates ownership across application boundaries rather than responsibility for a single technical layer. The project therefore provides evidence of full-stack engineering capability spanning frontend development, API architecture, persistence, authentication, application security, environment configuration and delivery.

Frontend Engineering

React Application Architecture

The application is structured as a React single-page application using page-level modules and reusable components.

The root application establishes the authentication provider before rendering the main application, allowing authentication state to be consumed throughout the application. Routing is handled through React Router, with public and authenticated sections separated through a reusable protected-route component.

The application contains dedicated modules for:

- Dashboard
- Tasks
- Projects
- Notes
- Goals
- Calendar
- Reminders
- Notifications
- Break functionality
- Global search
- Urgent items
- Alarms
- Account management

This architecture demonstrates practical experience decomposing a large application into feature-oriented modules rather than implementing the entire interface as a single component.

Technologies

- React
- React DOM
- React Router
- Vite
- JavaScript / ES Modules
- Tailwind CSS
- Framer Motion
- Lucide React
- Recharts

Authentication-Aware Frontend Architecture

Authentication is implemented through a centralized React context.

This application:

- Checks for a persisted authentication token
- Restores the current user from the backend
- Stores authenticated user information in application state.
- Persists the session in browser storage.
- Provides login and logout operations.
- Supports refreshing the authenticated user's information.
- Uses protected routes to prevent unauthenticated access to application modules.

The authentication context separates session management from individual pages, allowing authenticated state to be consumed through a reusable `useAuth()` abstraction.

This provides evidence of:

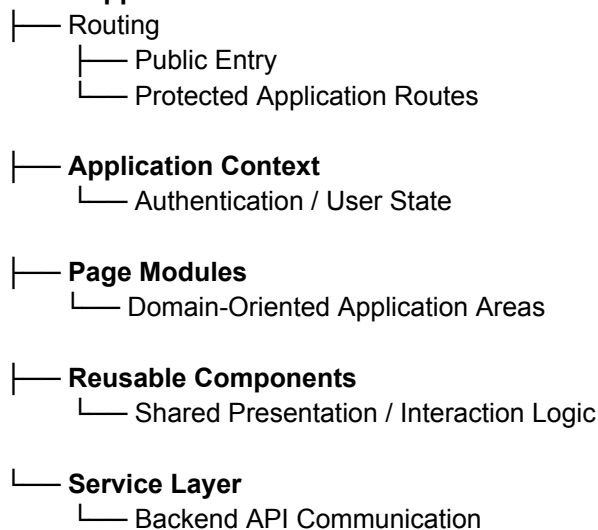
- React Context API
- Custom hooks
- Global application state
- Session restoration
- Authentication-aware rendering
- Client/server authentication coordination
- Protected application architecture

Frontend Application Architecture

The frontend operates as a client application responsible for presentation, navigation, authenticated session state, application-level state and communication with backend services.

Its architectural flow can be represented as:

React Application



The frontend maintains a deliberate separation between interface composition and persistence concerns. Components operate on application state and invoke service-layer operations rather than directly implementing backend communication throughout the interface.

This establishes a client-side architecture based on:

Presentation → State → Service Layer → API

rather than coupling interface components directly to server persistence.

The implementation also demonstrates derived application state, asynchronous resource loading, controlled forms, event-driven interactions, client-side validation and state reconciliation following server mutations.

Backend Engineering

Node.js and Express REST API

The backend is implemented using Node.js and Express.

The backend package configuration establishes:

- Express 5
- Mongoose
- JSON Web Token
- bcryptjs
- Multer
- CORS
- dotenv
- Nodemon

The application uses an API-oriented architecture in which frontend services communicate with backend REST endpoints.

Core HTTP operations include:

- GET
- POST
- PUT
- DELETE

The frontend service layer communicates with these endpoints through the browser Fetch API, including authenticated Bearer tokens.

Backend Application Architecture

The backend provides the server-side application boundary between the client and persistent data.

HTTP Request

→ Express Application
→ Route Handler
→ Authentication Middleware
→ Authenticated User Context
→ Domain Resource Operation
→ Mongoose Model
→ MongoDB
→ HTTP Response

The backend is responsible for enforcing the server-side boundary around persistent application resources. Client requests are translated into authenticated resource operations through Express routes and middleware before reaching the persistence layer.

The architecture incorporates:

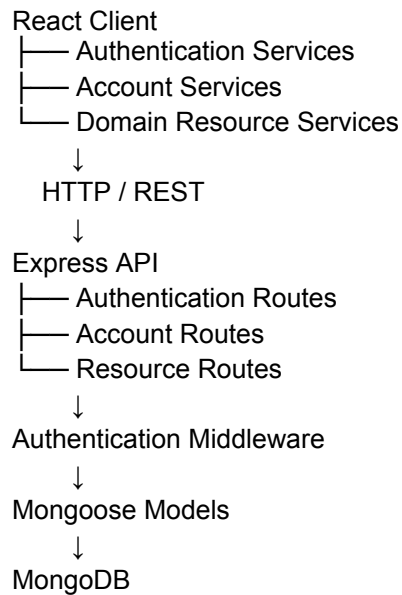
- RESTful resource endpoints
- HTTP method semantics
- Authentication middleware
- JWT verification
- User-scoped database queries

- Mongoose model operations
- JSON response handling
- Multipart request processing
- Concurrent database operations
- Account and workspace lifecycle operations

This creates a clear separation between client concerns and server-side resource management while providing a consistent API boundary for the React application.

API Architecture

The application uses REST as the primary communication boundary between the client application and server-side resources.



The API architecture separates transport concerns from frontend presentation and persistence concerns. HTTP requests carry the authenticated user context through Bearer tokens, while request payloads are serialized as JSON for structured operations or `multipart/form-data` where binary content is required.

The service layer therefore acts as the client-side integration boundary:

React Application → Service Module → REST Endpoint → Express Route → Persistence Layer

This provides a reusable mechanism for communicating with the backend while allowing frontend modules to remain independent of database implementation details.

Database Engineering

MongoDB + Mongoose

MongoDB provides persistent application storage, with Mongoose providing schema definitions and model-based access.

The application contains dedicated models for core productivity entities including:

- User
- Task
- Project
- Goal
- Note
- Reminder
- Alarm
- Notification

The Task schema, for example, establishes a required relationship to the authenticated user and defines its productivity-specific fields and related entity references.

Other models follow a similar pattern, supporting a consistent user-owned data architecture.

Authentication and Authorization Boundary

The application separates the concepts of establishing identity and restricting access to user-owned resources.

Credential Submission

→ Authentication

→ JWT Issuance

→ Client Session State

→ Bearer Token

→ Authentication Middleware

→ Authenticated User Identity

→ User-Scoped Resource Query

→ Persistent Data

Authentication establishes the identity represented by the request. The authenticated identity is then used by backend resource operations to constrain access to application data belonging to that user.

This creates an authorization boundary between:

Authenticated Identity → Permitted User-Owned Resources

The model is therefore not limited to frontend route protection. Access control is also represented at the backend resource layer, where authenticated user identity participates in database operations.

This distinction is important in a multi-user application because hiding an interface route does not constitute data authorization. The backend remains responsible for enforcing user-specific access to persistent resources.

User-Scoped Architecture

A key aspect of the backend is that productivity data is associated with individual users.

Authenticated requests derive the user identity from the authentication middleware and use that identity when querying application resources.

This enables a workspace model in which users interact with their own:

- Tasks
- Projects
- Goals
- Notes
- Reminders
- Alarms
- Notifications

rather than receiving a shared global dataset.

This is an important distinction between a simple demonstration application and an authenticated multi-user application architecture.

Authentication & Security

JWT Authentication

The backend implements JSON Web Token authentication.

On successful login, the server generates a JWT containing the authenticated user's identifier and assigns a defined expiration period.

The frontend subsequently sends the token using the HTTP Authorization Bearer scheme. Protected backend routes use authentication middleware to establish the authenticated user before performing user-specific operations.

This creates an end-to-end authentication chain:

Login → JWT issuance → client persistence → Bearer token → middleware verification → user-scoped database operation

Password Security

Passwords are not stored as plaintext.

The authentication backend uses bcryptjs to:

- Generate salts
- Hash passwords
- Compare supplied credentials against stored hashes.

Password verification is also used for sensitive account operations such as password changes and destructive workspace/account actions.

This demonstrates practical implementation of:

- Password hashing
 - Credential verification
 - Salt generation
 - Authenticated account operations
-

Account Lifecycle Management

The account module goes beyond basic profile editing.

The application supports:

- Profile updates
- Email updates
- Job/title information
- Bio management
- Avatar uploads
- Avatar removals
- Preference management
- Password changes
- Data exports
- Workspace data clearing
- Account deletion
- Logout

The frontend account implementation loads multiple workspace resources concurrently using `Promise.all`, including tasks, projects, goals, reminders, notes and alarms.

This demonstrates asynchronous orchestration across multiple API resources.

Profile and File Handling

Profile management includes avatar support.

The frontend can switch between JSON requests and `FormData` when an avatar file is supplied, allowing textual profile data and a binary upload to be submitted through the same profile workflow.

The backend uses Multer middleware for multipart avatar handling and filesystem operations for avatar replacement/removal.

This demonstrates experience with:

- Multipart form submission
- File upload middleware
- Browser `File` objects
- `FormData`
- Server-side file handling
- Profile asset lifecycle management

Data Export

The platform provides a user data export workflow.

The backend gathers the user's profile and productivity collections concurrently using `Promise.all`, returning a structured export payload.

The frontend converts the returned data into a JSON `Blob`, creates a browser object URL and triggers a client-side download before revoking the temporary URL.

This is a useful demonstration of both:

backend aggregation and **browser-side file generation/download handling**.

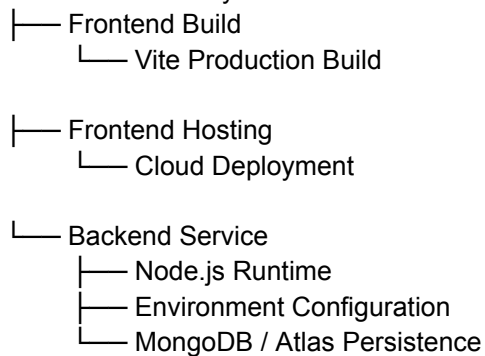
Application Delivery Architecture

The project separates development configuration from production deployment through environment-aware application configuration.

Local Development



Production Delivery



The frontend resolves its API endpoint through environment configuration, allowing deployment-specific infrastructure to be substituted without modifying application-level API logic.

This demonstrates an understanding of the distinction between **application code and deployment configuration**, which is fundamental to maintaining the same codebase across development and production environments.

Cloud & Deployment

The project was developed with environment-aware API configuration rather than hard-coding the production API endpoint into the application.

The frontend uses:

`VITE_API_URL`

with a localhost fallback for development.

This allows the same frontend codebase to communicate with different backend environments without changing application logic.

The project portfolio material identifies GitHub-based development and production deployment across modern cloud platforms, with the deployed frontend represented through Vercel.

Deployment / delivery skills demonstrated

- Git-based development
 - Environment variables
 - Vite production builds
 - Frontend cloud deployment
 - Backend deployment configuration
 - Development vs production API separation
 - Node.js production startup configuration
-

Development Tooling

The project uses a modern JavaScript development toolchain.

Frontend

- Vite
- ESLint
- React Hooks linting
- React Refresh
- npm scripts
- Tailwind CSS
- Framer Motion
- Lucide React
- Recharts

Backend

- Node.js
- Express
- Nodemon
- npm
- dotenv
- CORS

Version Control / Delivery

- Git
- GitHub
- Cloud deployment platforms

The project therefore demonstrates not only application coding but the supporting tooling required to develop, build and deliver a full-stack JavaScript application.

Engineering Architecture Principles

Several architectural principles are consistently represented throughout the implementation.

Separation of Concerns

The system separates:

React Presentation → Application State → API Services → Express Routes → Authentication Middleware → Mongoose Persistence

This prevents the frontend presentation layer from becoming directly coupled to database operations.

Reusable Application Boundaries

Authentication state, API communication and common interface behaviour are abstracted into reusable mechanisms rather than duplicated independently across application areas.

User-Scoped Persistence

Application resources are associated with authenticated users, establishing an ownership boundary within the persistence layer.

API-Driven Architecture

The React client interacts with persistent application data through REST endpoints rather than directly accessing the database.

Persistent Server-Side State

The backend and MongoDB provide the persistent source of application data, while React maintains the client representation required for responsive interaction.

Layered Security

Security is represented at multiple levels:

Credential Protection → JWT Authentication → Protected Routes → Authentication Middleware → User-Scoped Resource Access

Environment Separation

Runtime configuration is externalized through environment variables, separating deployment-specific infrastructure from application source code.

Modular Domain Architecture

The application is organized around multiple related domain entities while maintaining a common authenticated workspace and shared application infrastructure.

Together, these principles demonstrate architectural thinking beyond individual feature implementation and provide the foundation for extending the platform with additional application domains and services.

Engineering Problems & Solutions

1. Coordinating multiple application domains

Challenge

Tasks, projects, goals, notes, reminders, alarms and notifications needed to coexist within one application without becoming completely independent systems.

Solution

The application adopted a modular architecture with dedicated models, service layers, routes and frontend modules while maintaining common concepts such as user ownership, status, priority, due dates and linked entities.

This provided separation at the implementation level while allowing the product to operate as a single workspace.

2. Maintaining authenticated application status

Challenge

Authentication needed to survive navigation and browser refreshes while remaining available throughout protected application modules.

Solution

A centralized React authentication context was combined with persistent token storage and backend user retrieval.

A reusable protected-route component then became the authentication boundary for private application routes.

3. Keeping frontend state synchronized with backend mutations

Challenge

CRUD operations should immediately update the interface without forcing a complete page reload.

Solution

Mutation handlers update local React state using the server's returned representation.

For example, completing a task replaces the corresponding task in the local collection with the updated API response.

This creates a responsive client experience while retaining the backend as the source of persistent truth.

4. Managing complex task states

Challenge

A task could simultaneously have completion, status, priority, due-date and interaction metadata, making a simple completed/not-completed model insufficient..

Solution

The task domain was expanded into explicit status, completion, priority, category, due-date, flag, like and comment properties.

Frontend business rules then derive states such as **Overdue** from persisted task data.

5. Handling interconnected productivity entities

Challenge

The application needed to represent relationships between different types of productivity objects.

Solution

The data model incorporates linked entity fields and Mongoose references across productivity models. The task schema, for example, includes references to projects, tasks, goals, notes and reminders.

6. Managing destructive operations

Challenge

Actions such as deleting tasks, clearing workspace data and deleting accounts are irreversible.

Solution

The frontend uses dedicated confirmation workflows for destructive operations, while sensitive account operations also require credential verification on the backend.

Project Impact

The primary impact of the project is technical rather than measured through commercial KPIs. It demonstrates the ability to independently engineer a **complete full-stack application spanning the entire software delivery stack**.

The project provides concrete evidence of capability in:

Frontend

React application development, routing, component architecture, state management, form handling, interactive interfaces and client-side data processing.

Backend

Node.js, Express, REST API development, middleware, authentication, account lifecycle management and asynchronous server operations.

Database

MongoDB, Mongoose schemas, user ownership and interconnected document-oriented data models.

Security

JWT authentication, Bearer-token authorization, password hashing and credential verification.

Cloud

Environment-aware configuration, production builds and cloud deployment.

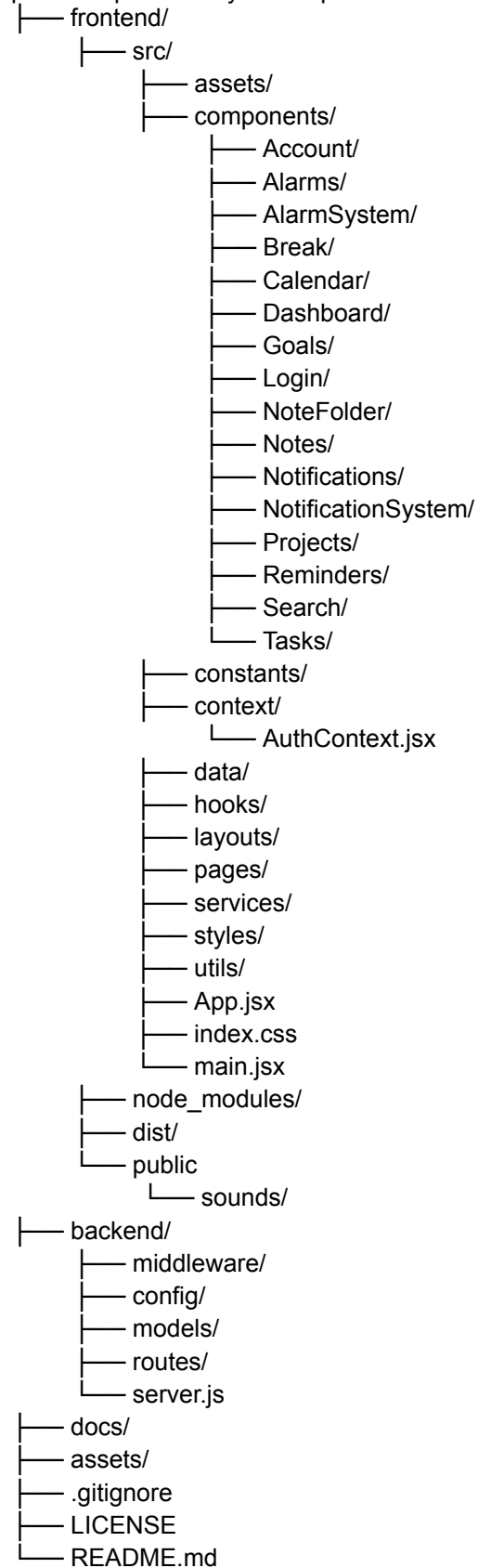
Architecture

Separation of frontend presentation, service/API communication, backend routes, middleware and persistence models.

The project portfolio description similarly positions the application around full-stack architecture, REST APIs, database design, authentication, protected routes, state management, CRUD and deployment.

Repository Structure

personal-productivity-desktop



Technical Skills Demonstrated

Frontend Engineering

- React
- React Hooks
- React Context API
- Custom hooks
- React Router
- Protected routes
- Component architecture
- Controlled forms
- State management
- Client-side validation
- Event handling
- DOM references
- Lifecycle management
- Search
- Filtering
- Sorting
- Derived state
- Modal Workflows
- Reusable overlays
- Responsive web application development

Backend Engineering

- Node.js
- Express
- REST API development
- Middleware
- HTTP methods
- JSON APIs
- Authentication middleware
- Asynchronous programming
- Concurrent operations with Promise.all
- CRUD operations
- Account lifecycle APIs
- File upload handling

Database Engineering

- MongoDB
- MongoDB Atlas
- Mongoose
- Schema design
- Document modelling
- ObjectId relationships
- User-scoped data
- Cross-entity references
- Persistent CRUD operations

Authentication & Security

- JWT
- Bearer authentication
- Password hashing
- bcrypt
- Credential verification
- Protected routes
- Session restoration
- Sensitive field exclusion
- Destructive-operation verification

APIs & Integration

- REST APIs
- Fetch API
- JSON serialization
- JSON parsing
- HTTP status handling
- Authorization headers
- Multipart/form-data
- Environment-based API configuration

Cloud & Delivery

- Git
- GitHub
- Vite production builds
- Environment variables
- Cloud deployment
- Vercel
- Node.js production startup

Development

- JavaScript
 - ED Modules
 - Npm
 - ESLint
 - Debugging
 - Component decomposition
 - Separation of concerns
-

Technology Stack

Layer Technologies

Frontend React, React DOM, Vite

Routing React Router

Styling Tailwind CSS, modular CSS

Animation / Interaction Framer Motion, CSS transitions

Icons Lucide React

Data Visualisation Recharts

Backend Node.js, Express

Database MongoDB, MongoDB Atlas, Mongoose

Authentication JWT, bcryptjs

File Handling Multer, browser FormData

API Communication REST, Fetch API

Configuration dotenv, Vite environment variables

Middleware / Infrastructure CORS, Express middleware

Development npm, Nodemon, ESLint

Version Control Git, GitHub

Deployment Vercel / cloud deployment infrastructure

What Could Be Improved

The project provides strong evidence of full-stack capability, but there are several areas that would be appropriate for a future production-hardening iteration.

Security hardening

The account implementation contains password-reset and user-deletion endpoints whose authentication model should be strengthened for a production system.

A production implementation should use a secure password-reset workflow based on short-lived, single-use reset tokens rather than relying solely on an email address and replacement password.

Centralised API error handling

Some frontend service functions perform more comprehensive HTTP error checking than others.

A future iteration could introduce a shared API client that standardises:

- Token handling
- HTTP error processing
- JSON parsing
- Authentication failures
- Network errors
- API response normalization

This would reduce duplicated request logic.

Data integrity and deletion semantics

Because multiple productivity entities reference one another, production hardening should introduce a deliberate strategy for relationship cleanup and referential integrity when users or entities are deleted.

For example, cascading deletion rules could be centralized rather than handled independently within individual routes.

Validation

The frontend provides useful client-side validation, but production-quality validation should be consistently enforced at the API boundary as well.

Schema-level and route-level validation could be expanded with explicit constraints for:

- Allowed statuses
- Allowed priorities
- Categories
- Date formats
- Field lengths
- Relationship identifiers
- Request payload structure

Automated testing

A significant future improvement would be a dedicated testing layer covering:

- Authentication
- Protected routes
- CRUD APIs
- Account lifecycle operations
- Task-state transitions
- Validation
- Database interactions
- Critical React workflows

This would provide stronger regression protection as the platform expands.

Scalability

The current frontend performs several client-side filtering and sorting operations. For a substantially larger dataset, pagination, server-side filtering and indexed database queries could reduce client-side processing.

Similarly, frequently accessed dashboard aggregations could eventually be optimized through purpose-built queries or aggregation pipelines.

Overall Engineering Capability

This project demonstrates the ability to move beyond isolated coding exercises and build a **complete, persistent, authenticated software system**.

The strongest evidence is the breadth of the implementation:

React frontend

- **protected application architecture**
- **service/API layer**
- **Node.js / Express REST backend**
- **JWT authentication**
- **bcrypt password security**
- **MongoDB / Mongoose persistence**
- **interconnected domain models**
- **account and data lifecycle management**
- **cloud deployment**

The project therefore demonstrates practical capability across **frontend engineering, backend engineering, database development, API design, authentication, application architecture and deployment**.

Rather than presenting the project simply as a "to-do app," it should be positioned as a **full-stack productivity platform** that demonstrates the engineering required to build and connect multiple application domains into one authenticated system.

Portfolio Summary

Full-Stack Productivity Platform

Full-Stack Web Application · React · Node.js · Express · MongoDB · Mongoose · JWT

Engineered a full-stack productivity platform that unifies tasks, projects, goals, notes, reminders, alarms, calendar workflows, notifications and account management within a persistent authenticated workspace. Implemented a React/Vite frontend with protected routing, reusable component architecture, complex state management, search, filtering and sorting; a Node.js/Express REST API with JWT authentication and bcrypt password security; and a MongoDB/Mongoose data layer supporting user-scoped and interconnected productivity entities. The project also includes account lifecycle management, profile/avatar handling, data export, destructive-operation safeguards, environment-based configuration and cloud deployment.

Core engineering strengths demonstrated: Full-Stack Development · React · Node.js · Express · MongoDB · Mongoose · REST APIs · JWT Authentication · Password Security · Protected Routes · CRUD · Database Design · State Management · API Integration · Cloud Deployment · Git/GitHub.
